```
    ➥ model.NumberOfGuests;
    var result = new  { TotalCost = totalCost };                    ❷
    return new JsonResult(result);                                  ❸
}
```

❶ Change the return type to JsonResult and add the [FromBody] attribute to the handler parameter to tell the framework where to find the JSON data.

❷ Create an object to represent the returned data.

❸ Pass the object to the JsonResult constructor, serializing it appropriately.

Listing 11.16 shows the plain JavaScript version of the button click event handler that generates the JSON, posts it to the server, and processes the result. There are some essential points to note. You must pass the request verification token in the header this time. You cannot include it within the JSON in the request body because request verification takes place before the framework parses the JSON. In addition, you must specify the content type of the request as `application/json`. Finally, when you use the Fetch API, you use the `json()` method on the response (as opposed to the `text()` method you used previously) to deserialize the response, so you can work with it. The default JSON serializer generates property names using camel case, so the `TotalCost` property of the anonymous object you passed to the `JsonResult` constructor becomes `totalCost`.

Listing 11.16 The button click handler that posts JSON using the Fetch API

```
document.querySelector('.btn-success')
    .addEventListener('click', () => {
        const token = document.querySelector(
        ➥ '[name="__RequestVerificationToken"]').value;
        const data = {
                startdate: document.getElementById('StartDate').value,
                enddate: document.getElementById('EndDate').value,
                numberofguests: document.getElementById(
                ➥ 'NumberOfGuests').value,
                property: {
                    dayrate: document.getElementById(
                    ➥ 'Property_DayRate').value
```